

SCAN ME



UNIT – II	10 Hours
Regular Expression and Languages: Regular Expressions, Finite Automata and Regular Expressions, Applications of Regular Expressions, Properties of Regular Languages: Pumping Lemma for Regular Languages, Closure properties of Regular Languages, Equivalence and Minimization of Automata <i>Textbook 1: Chapter 3.1,3.2,3.3,4.1,4.2,4.3,4.4</i>	

- **Regular Expression and Languages:**
 - Regular Expressions,
 - Finite Automata and Regular Expressions,
- **Applications of Regular Expressions,**
- **Properties of Regular Languages:**
 - Pumping Lemma for Regular
 - Languages,
 - Closure properties of Regular Languages,
 - Equivalence and Minimization of Automata

- A Regular Expression (regex or regexp) is a sequence of characters that define a search pattern, mainly used for string pattern matching or manipulation.
- Regular expressions are often employed in programming languages, text processing tools, and search algorithms to search, edit, or manipulate text based on specific patterns.

- **Definition**

A sequence of characters that specifies a match pattern in text for various operations like find, find and replace, or input validation.

- **Abbreviated as**

Regex or Regexp, and sometimes referred to as rational expression.

- **Origin**

The concept was formalized in the 1950s by American mathematician Stephen Cole Kleene.

- **Common Use**

Widely used in search engines, word processors, text editors, text processing utilities like sed and AWK, and programming languages.

- **Syntax Variations**

Different syntaxes for writing regular expressions exist, including the POSIX standard and the Perl syntax.

Regular expressions:

To simplify the notations of regular languages, and drawing analogy to arithmetic expressions used in algebra, we could replace the union symbol $\cup$ with the plus sign $+$, drop the braces “{” and “}” for sets, and use parentheses “(“ and “)” for grouping when necessary, such as the following:

Example. The following are some regular expressions over alphabet $A = \{a, b\}$, and the corresponding regular languages:

<u>Expression</u>	<u>Language</u>
$a + ab$	$\{a, ab\}$
$(a + b)^*bb$	$\{a, b\}^*\{bb\}$
$ba(a + b)^*ab$	$\{ba\}\{a, b\}^*\{ab\}$
ε	$\{\varepsilon\}$

- **Regular Expressions** are a subset of formal languages specifically used to define Regular Languages, which can be recognized by finite automata.
- **Formal Languages**, as a broader concept, include Regular, Context-Free, Context-Sensitive, and Recursively Enumerable languages, each with increasing computational complexity and power

Feature	Regular Expressions	Formal Languages
Definition	A sequence of characters defining a search pattern used for string matching or manipulation.	A set of strings (words) formed using an alphabet, governed by specific rules or grammar.
Purpose	Pattern matching, searching, and text processing.	Defining and categorizing strings that belong to a particular language.
Type of Language	Describes Regular Languages (the simplest type of formal language in the Chomsky hierarchy).	Encompasses different language types: Regular, Context-Free, Context-Sensitive, and Recursively Enumerable.
Grammar Representation	Regular expressions are equivalent to Regular Grammars, which have rules in the form: $A \rightarrow aB$ or $A \rightarrow a$.	Languages can be defined by grammars: Regular Grammars, Context-Free Grammars, Context-Sensitive Grammars, or Unrestricted Grammars.
Finite Automaton	Can be represented by a Deterministic Finite Automaton (DFA) or Non-deterministic Finite Automaton (NFA).	Different types of automata represent languages: DFA/NFA for Regular Languages, Pushdown Automata for Context-Free Languages, Turing Machines for recursively enumerable languages.
Complexity	Limited to describing Regular Languages (cannot capture all possible patterns, e.g., nested structures).	More complex formal languages can describe more complicated patterns, including recursive or hierarchical structures.
Examples	a^+b^+ , $[a-zA-Z0-9]^+$ (Regular Expression matching alphanumeric strings).	Formal languages: Regular (a^+b^+), Context-Free (balanced parentheses: $(a^n b^n)$), etc.
Use Cases	Text search, input validation, data extraction (e.g., validate email addresses, search in documents).	Defining the syntax of programming languages, compilers, and natural language processing (NLP).
Power	Can describe only regular languages, which are less powerful than context-free or context-sensitive languages.	Formal languages vary in power, with recursively enumerable languages being the most powerful.
Limitations	Cannot describe patterns with nested or recursive structures (e.g., balanced parentheses).	Higher classes of formal languages (like context-free or context-sensitive) can describe nested and recursive patterns.
Hierarchy	Forms part of the Chomsky hierarchy as Regular Languages.	Includes all types in the Chomsky hierarchy: Regular, Context-Free, Context-Sensitive, and Recursively Enumerable.

Arden's Theorem is a fundamental result in the theory of formal languages and automata, particularly in the context of regular expressions and finite automata.

- It provides a method to solve equations involving regular expressions, which is useful when converting finite automata to regular expressions

Statement of Arden's Theorem:

Let P and Q be regular expressions over some alphabet, and let X be an unknown regular expression.

If the equation is of the form:

$$X = Q + X \cdot P \quad (R = Q + RP)$$

then the solution is:

$$X = Q \cdot P^* \quad (R = QP^*)$$

provided that P does not contain the empty string (ϵ)

X is the regular expression we want to solve for.

P and Q are known regular expressions.

The equation $X = Q + X \cdot P$ represents the language X as being either:

Statement of Arden's Theorem:

Let P and Q be regular expressions over some alphabet, and let X be an unknown regular expression.
If the equation is of the form:

$$X = Q + X \cdot P \quad (R = Q + RP)$$

then the solution is:

$$X = Q \cdot P^* \quad (R = QP^*)$$

provided that P does not contain the empty string (ϵ)

- X is the regular expression we want to solve for.
- P and Q are known regular expressions.

The equation $X = Q + X \cdot P$ represents the language X as being either:

- Q (a set of strings described by Q) or
- $X \cdot P$ (concatenation of X with P, which generates further strings).

- Arden's Theorem allows you to solve this recursive equation by expressing X as the concatenation of Q with P 's Kleene star (which represents zero or more repetitions of P).
- P should not have the empty string ϵ as a member. If P contains ϵ , the theorem doesn't apply directly.
- Proof Outline:
- Given the equation $X = Q + X \cdot P$, we want to isolate X :
- X is present on both sides, so substitute $Q + X \cdot P$ into X recursively.
- Keep substituting until you get a form where X is only in terms of Q and P (which leads to a series expansion).
- This results in the expression $X = Q \cdot P^*$,
where P^* captures all possible repetitions of P

Proof of Arden's Theorem: $(R=Q+RP)$ has a unique solution i.e $R=QP^*$

$$R=Q+RP \text{-----(1)}$$

$$= Q+(QP^*)P \quad \text{insert } R=QP^*$$

$$= Q(E+P^*P) \quad [I \equiv E+R^*R=R^*]$$

$$R = QP^* \text{ is Proved}$$

$$R=Q+RP$$

$$= Q+(Q+RP)P \quad \text{insert } R=QP$$

$$= Q+QP+RP^2$$

$$= Q+QP+(Q+RP)P^2$$

$$Q+QP+QP^2+RP^3$$

.....

$$= Q+QP+QP^2+\dots+QP^n+RP^{n+1} \quad \text{insert : } R=QP^*$$

$$= Q+QP+QP^2+\dots+QP^n+QP^*P^{n+1}$$

$$= Q[E+P+P^2+\dots+P^n+P^*P^{n+1}]$$

$$= QP^*$$

Example:

Suppose you have the equation:

$$X = a + Xb$$

Here, $Q = a$ and $P = b$. Using Arden's Theorem:

$$X = a \cdot b^*$$

Thus, the solution is $X = ab^*$,
which represents the language of strings starting with a followed by zero or more b's.

Arden's Theorem is widely used to convert **finite automata** into **regular expressions**. It simplifies the process of solving for regular expressions that describe the language accepted by a finite automaton.

List what are the Operation are performed on Alphabets to obtain regular expression

- Operations performed on alphabets to obtain a regular expression are based on constructing new patterns from basic elements (alphabets or symbols).
- The following operations are essential in defining regular expressions:

1. Union (OR)

- Symbol: + or ,|
- Definition: The union of two regular expressions represents the set of strings that match either one or both of the expressions.
- Example: If $R1 = a$ and $R2 = b$, then $R1 + R2 = a + b$ matches either a or b .
- PUQ , $P+Q$, or (P,Q) are equivalent.

2. Concatenation

- Symbol: Implicit (juxtaposition of expressions)
- Definition: The concatenation of two regular expressions represents the set of strings obtained by concatenating a string from the first expression with a string from the second expression.
- Example: If $R1 = a$ and $R2 = b$, then $R1 \cdot R2 = ab$ matches the string "ab".
 - $P=\{ab,abb\}$ $q=\{a,aa,bb\}$
 - $PQ=\{aba, abaa, abbb, abba, abbaa, abbbb\}$

3. Kleene Star (Closure)

Symbol: *

- Definition: The Kleene star of a regular expression represents the set of strings obtained by repeating the expression zero or more times (including the empty string).

- Example:

If $R = a$,

then $R^* = a^*$

matches the empty string, "a", "aa", "aaa", and so on.

- The Closure of a Language L is denoted by L^*

4. Kleene Plus

- Symbol: +
- Definition: Similar to the Kleene star but requires at least one occurrence of the regular expression.
- Example: If $R = a$, then $R^+ = a^+$ matches "a", "aa", "aaa", but not the empty string.

5. Optional (Zero or One Occurrence)

Symbol: ?

Definition: This operation represents the option of having zero or one occurrence of the regular expression.

Example: If $R = a$, then $R? = a?$ matches either the empty string or "a".

6. Character Set (Choice)

Symbol: []

Definition: Defines a set of characters, where any one character from the set can match.

Example: [abc] matches either "a", "b", or "c".

7. Range

Symbol: - (inside square brackets)

Definition: A shorthand for specifying a range of characters.

Example: [a-z] matches any lowercase letter between "a" and "z".

8. Complement (Negation)

Symbol: [^]

Definition: Matches any character that is not in the set.

Example: [^abc] matches any character except "a", "b", or "c".

9. Empty String (Epsilon)

- Symbol: ϵ or sometimes represented by the empty string "".
- Definition: Represents a regular expression that matches the empty string.
- Example: ϵ matches the empty string.

10. Precedence Grouping

- Symbol: ()
- Definition: Parentheses are used to group parts of regular expressions, controlling the precedence of operations.
- Example: $(a + b)^*$ matches any combination of "a"s and "b"s, including the empty string.

Operation	Symbol	Description	Example
Union (OR)	+ or `	`	Matches either one or both regular expressions
Concatenation	Juxtaposition	Matches sequences formed by two regular expressions	ab (matches ab)
Kleene Star	*	Zero or more repetitions of a regular expression	a* (matches `, a, aa, ...)
Kleene Plus	+	One or more repetitions of a regular expression	a+ (matches a, aa, ...)
Optional (Zero or One)	?	Zero or one occurrence of a regular expression	a? (matches a or `)
Character Set	[]	Matches any one character from a set	[abc] (matches a, b, c)
Range	- (in [])	Matches any character within the specified range	[a-z] (matches a, b, ..., z)
Complement	[^]	Matches any character not in the set	[^abc] (matches anything except a, b, c)
Empty String	ϵ or ""	Matches the empty string	ϵ (matches `)
Grouping	()	Groups expressions to control precedence	(ab)+ (matches ab, abab, ...)

These properties are fundamental when manipulating regular expressions and understanding the nature of regular languages.

- **Closure**: Union, concatenation, Kleene star, intersection, complement, and difference.
- **Associativity, commutativity** (only for union), and **distributivity**.
- Identity and annihilation rules.
- Idempotence of union and Kleene star.
- De Morgan's laws for regular languages.

1. Closure Properties

Regular languages, which are described by regular expressions, are closed under the following operations. This means that applying these operations on regular languages produces another regular language.

Union: If L_1 and L_2 are regular languages, then $L_1 \cup L_2$ (union) is also a regular language.

Example: If $L_1 = \{a, b\}$ and $L_2 = \{c, d\}$, then $L_1 \cup L_2 = \{a, b, c, d\}$.

Concatenation: If L_1 and L_2 are regular languages, then their concatenation L_1L_2 is also a regular language.

Example: If $L_1 = \{a, b\}$ and $L_2 = \{c\}$, then $L_1L_2 = \{ac, bc\}$.

Kleene Star: If L is a regular language, then L^* (the set of zero or more repetitions of strings from L) is also a regular language.

Example: If $L = \{ab\}$, then $L^* = \{\epsilon, ab, abab, ababab, \dots\}$.

Intersection: If L_1 and L_2 are regular languages, then $L_1 \cap L_2$ (intersection) is also a regular language.

Example: If $L_1 = \{a, b\}$ and $L_2 = \{b, c\}$, then $L_1 \cap L_2 = \{b\}$.

Complement: If L is a regular language, then the complement of L ($\Sigma^* \setminus L$, the set of all strings not in L) is also a regular language.

Example: If $L = \{a, b\}$, then the complement of L in alphabet $\{a, b, c\}$ is $L' = \{c\}$.

Difference: If L_1 and L_2 are regular languages, then $L_1 \setminus L_2$ (the difference) is also a regular language.

Example: If $L_1 = \{a, b\}$ and $L_2 = \{b\}$, then $L_1 \setminus L_2 = \{a\}$.

2. Associativity

- Union Associativity: The order of unions does not matter: $(L1 \cup L2) \cup L3 = L1 \cup (L2 \cup L3)$.
- Concatenation Associativity: The order of concatenation does not matter: $(L1L2)L3 = L1(L2L3)$.

3. Commutativity

- Union Commutativity: Union is commutative: $L1 \cup L2 = L2 \cup L1$.
- Concatenation is not commutative: In general, concatenation is not commutative: $L1L2 \neq L2L1$ (except in special cases).

4. Distributivity

- Concatenation distributes over union:
- $L1(L2 \cup L3) = L1L2 \cup L1L3$
- $(L1 \cup L2)L3 = L1L3 \cup L2L3$

5. Identity Properties

- Identity for Union: $L \cup \emptyset = L$ and $L \cup \Sigma^* = \Sigma^*$
- Union with the empty set leaves the language unchanged.
- Identity for Concatenation: $L\varepsilon = \varepsilon L = L$
- Concatenation with the empty string leaves the language unchanged.

6. Annihilation Properties

- Annihilation for Concatenation: $L \emptyset = \emptyset L = \emptyset$
- Concatenating a language with the empty set results in the empty set.

7. Idempotence

- Union Idempotence: $L \cup L = L$
- Union of a language with itself does not change the language.
- Kleene Star Idempotence: $L^{**} = L^*$
- Repeating the Kleene star operation on a language does not change it.

8. Null and Unit Rules

- Empty Set Rules:
- $\emptyset^* = \{\epsilon\}$ (zero or more repetitions of nothing is the empty string).
- $L \cup \emptyset = L$ (a union with the empty set does not change the language).
- Empty String Rules:
- $\epsilon^* = \{\epsilon\}$ (the Kleene star of the empty string is just the empty string).
- $L\epsilon = \epsilon L = L$ (concatenating with the empty string does not change the language).

9. Kleene Star Properties

- Kleene Star of a Set: If L is a regular language, then L^* is also a regular language and consists of strings that are the concatenation of zero or more strings from L .
- Example: If $L = \{ab\}$, then $L^* = \{\epsilon, ab, abab, ababab, \dots\}$.

10. Regular Expressions over Empty String ϵ

- If $L = \epsilon$, then:
- $L^* = \{\epsilon\}$ (repeated application of ϵ gives ϵ).
- $L + \epsilon = \epsilon$ (union of ϵ with itself is ϵ).

11. De Morgan's Laws

For regular languages, the De Morgan's laws apply, as they do for sets:

- $\Sigma^* \setminus (L1 \cup L2) = (\Sigma^* \setminus L1) \cap (\Sigma^* \setminus L2)$
- $\Sigma^* \setminus (L1 \cap L2) = (\Sigma^* \setminus L1) \cup (\Sigma^* \setminus L2)$

12. Equivalence of Regular Expressions and Finite Automata

- Every regular expression can be converted into an equivalent finite automaton (DFA or NFA), and every finite automaton can be described by a regular expression.

Identities of Regular Expressions

$$I_1 : \phi + R = R$$

$$I_2 : \phi R = R\phi = \phi$$

$$I_3 : \epsilon R = R\epsilon = R$$

$$I_4 : \epsilon^* = \epsilon \text{ and } \phi^* = \epsilon$$

$$I_5 : R + R = R$$

$$I_6 : R^* R^* = R^*$$

$$I_7 : RR^* = R^* R$$

$$I_8 : (R^*)^* = R^*$$

$$I_9 : \epsilon + RR^* = R^* = \epsilon + R^* R$$

$$I_{10} : (PQ)^* P = P(QP)^*$$

$$I_{11} : (P+Q)^* = (P^*Q^*)^* = (P^* + Q^*)^*$$

$$I_{12} : (P+Q)R = PR + QR \quad \text{and} \quad R(P+Q) = RP + RQ$$

Regular Expression -Example

- | | | |
|--------------------------|----------------------|-----------------|
| 1. {0,1,2} | 0 or 1 or 2 | $R=0+1+2$ |
| 2. {^,ab} | | $R=^ab$ |
| 3. {abb,a,b,bba} | abb or a or b or bba | $R=abb+a+b+bba$ |
| 4. {^,0,00,000,.....} | Closure of 0 | $R=0^*$ |
| 5. {1,11,111,1111,.....} | | $R=1^+$ |

An Example Proof using Identities of Regular Expression Prove that
 $(1+00^*1) + (1+00^*1) (0+10^*1)^* (0+10^*1)$ equal to $0^*1 (0+10^*1)^*$

$$\begin{aligned}
 \text{LHS} &= (1+00^*1) + (1+00^*1) (0+10^*1)^* (0+10^*1) \\
 &= (1+00^*1) [E + (0+10^*1)^* (0+10^*1)] && \text{from [I9=E+R*R=R*]} \\
 &= (1+00^*1) [0+10^*1]^* && \text{from [I3= E.R=R]} \\
 &= [E.1+00^*1] [0+10^*1]^* \\
 &= [E+00^*] 1 [0+10^*1]^* \\
 &= 0^*1[0+10^*1]^* \\
 &= \text{RHS}
 \end{aligned}$$

Design a Regular Expression for the Following Languages over {a,b}

1. Language accepting strings of length exactly 2
2. Language accepting strings of length at least 2
3. Language accepting strings of length at atmost 2

1) $L1 = \{aa, ab, ba, bb\}$

$R = aa + ab + ba + bb$

$= a(a+b) + b(a+b)$

$= (a+b)(a+b)$

2) $L2 = \{aa, ab, ba, bb, aaa, \dots\}$

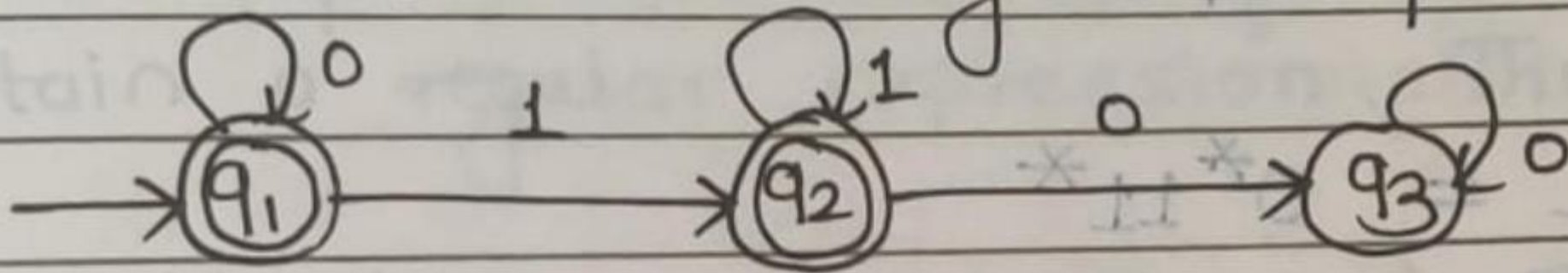
$R = (a+b)(a+b)(a+b)^*$

3) $L3 = \{E, a, b, aa, ab, ba, bb\}$

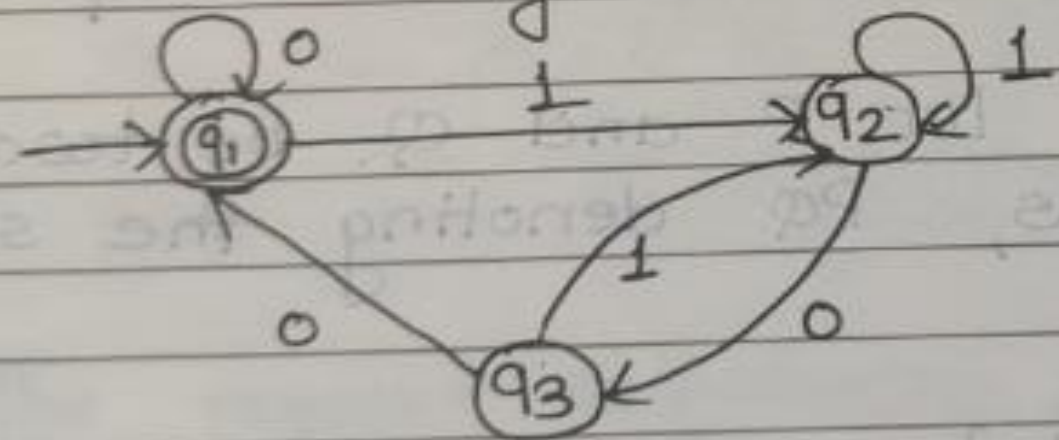
$R = E + a + b + aa + ab + ba + bb$

$= (E + a + b)(E + a + b)$

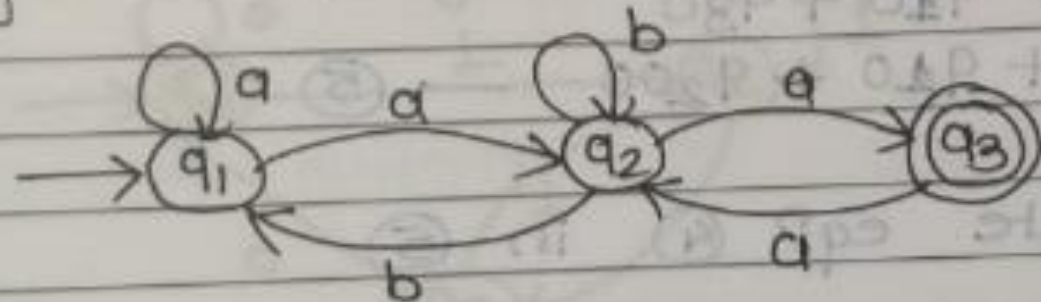
Q. Convert FA into regular expression



Q. Construct Regular expression corresponding to state diagram.

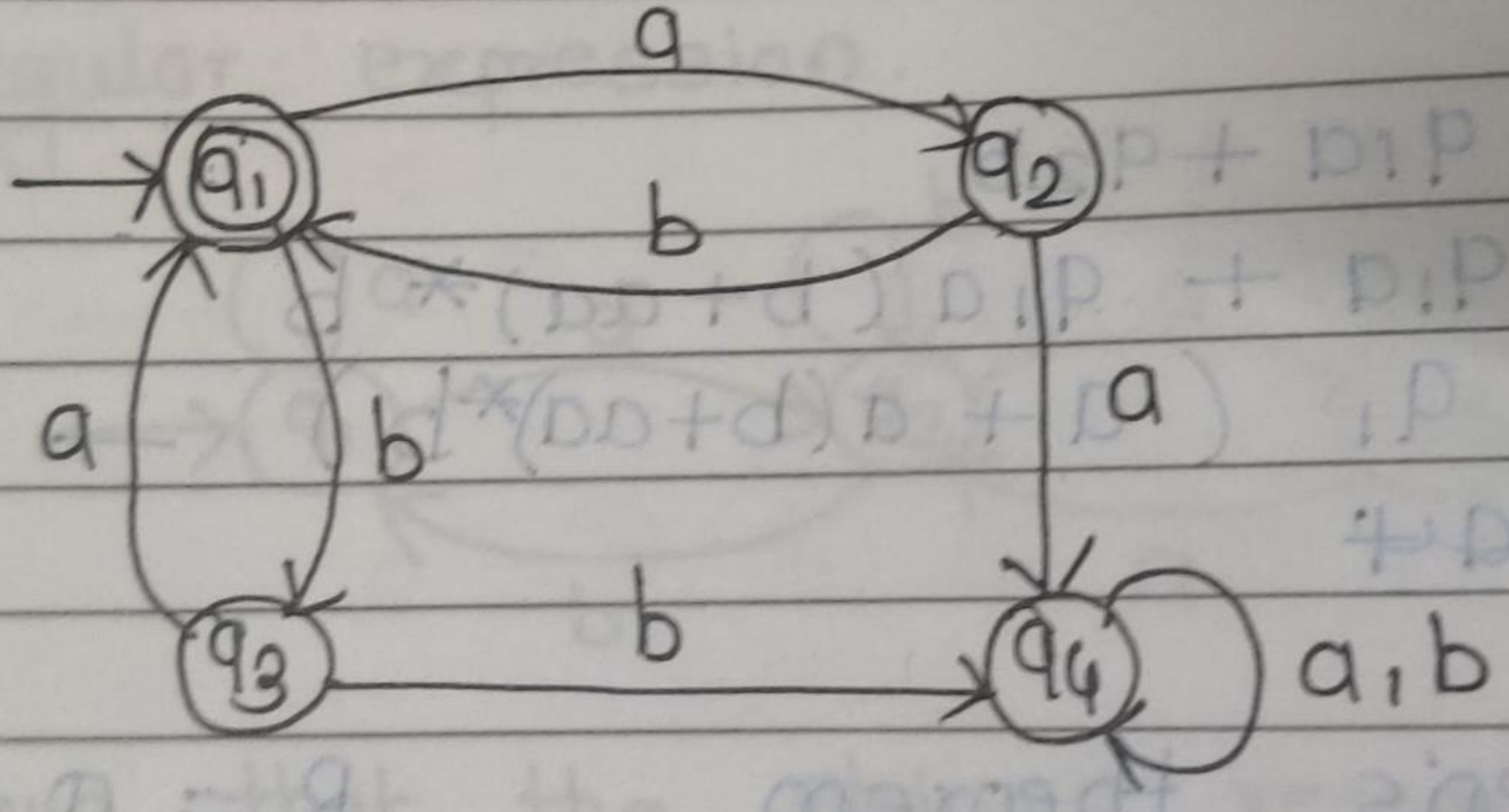


Q. Consider a transition system as in fig. Find the regular expression.

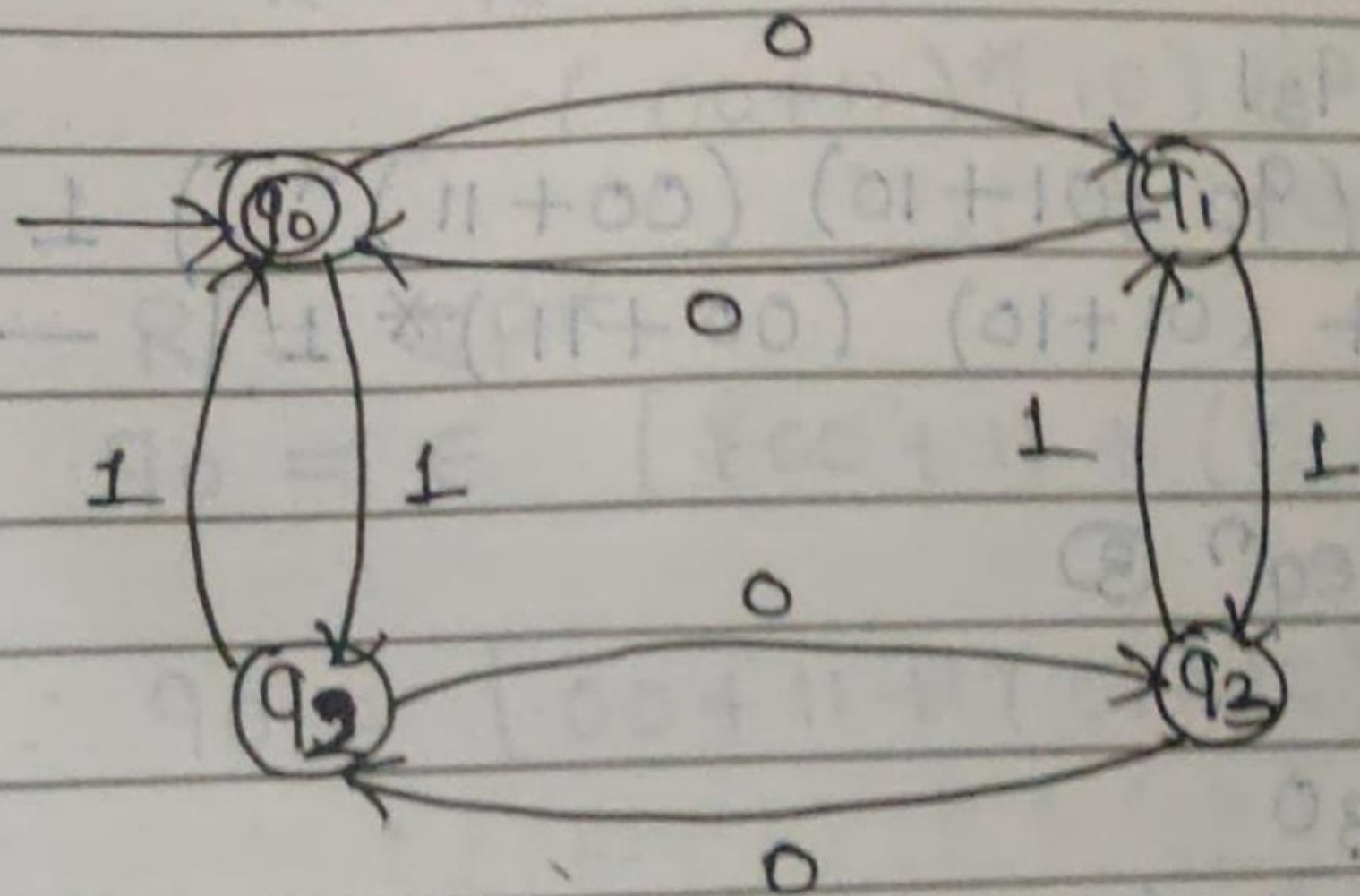


Prove that the strings recognized are
 $(a + (a(b+aa)^*b))^* a (b+aa)^* a$

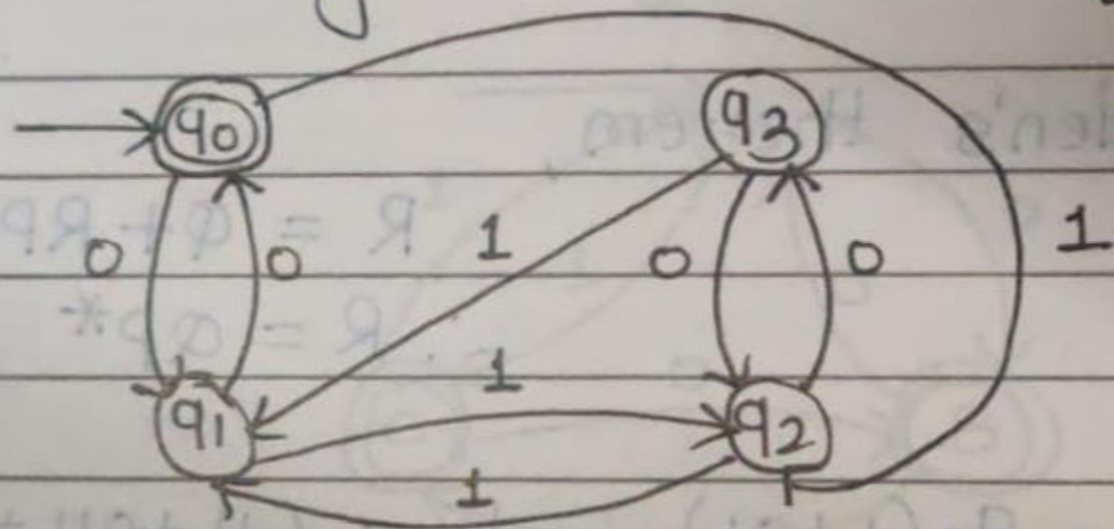
Q. Construct the FA into RE.



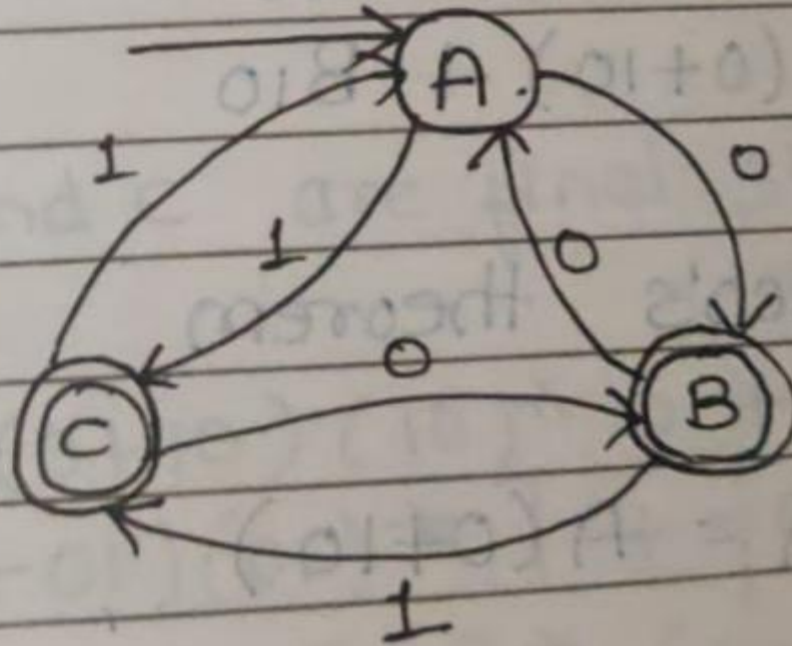
Construct RE for following state diagram



Construct Regular expression for given diagram!



Construct the regular expression for following DFA.



Applications of Regular Expressions

Regular expressions (regex) are powerful tools used in various fields for string manipulation and pattern matching. Their versatility makes them essential in programming, data processing, and text analysis. Here are some key applications:

Applications of Regular Expression:

- 1. Text Search and Manipulation**
- 2. Input Validation**
- 3. Data Extraction and Parsing**
- 4. Programming and Code Parsing**
- 5. Web Development**
- 6. Network Security**
- 7. Command-Line Operations**
- 8. Database Operations**

Text Searching and Matching

- Finding Patterns:** Regex is used to find specific patterns in large texts, such as extracting dates, email addresses, phone numbers, URLs, and more.
- Keyword Matching:** Used in search engines, text editors, and word processors to find particular words or phrases quickly.

Data Extraction and Parsing

Log Analysis: Extracts specific information from log files such as IP addresses, error codes, or timestamps.

Web Scraping: Pulls particular data (e.g., extracting HTML tags, parsing specific data patterns) from web pages.

File Parsing: Analyzes and pulls structured information from unstructured file formats like CSV, JSON, or XML.

Text Transformation and Formatting

String Replacement: Replaces substrings within a text (e.g., replacing HTML tags, updating keywords in bulk).

Reformatting Text: Converts text from one format to another, like swapping date formats (e.g., MM/DD/YYYY to YYYY-MM-DD).

Syntax Highlighting

- Code Editors: Highlights syntax in code editors like VS Code, Sublime, or Atom by recognizing keywords, variable names, and language-specific patterns.
- Markup Language Rendering: Helps render LaTeX, Markdown, or similar markup languages for web displays.

Search and Replace Operations in IDEs

- Regex is extensively used in Integrated Development Environments (IDEs) for global search and replace tasks, particularly in codebases, to make structural changes quickly.

Natural Language Processing (NLP)

- Used to identify and process specific patterns in text data, such as extracting named entities, finding sentence boundaries, or tokenizing text.

Security and Log Monitoring

- Analyzes and filters security logs to detect suspicious activity patterns, such as failed login attempts or identifying specific user agents.

Configuration File Management

Regex is used in system configuration tools like grep, sed, awk, and Perl scripts for finding and replacing configurations in system files.

https://beansoftware.com/ASP.NET-Tutorials/Validation-Regular-Expressions.aspx#google_vignette



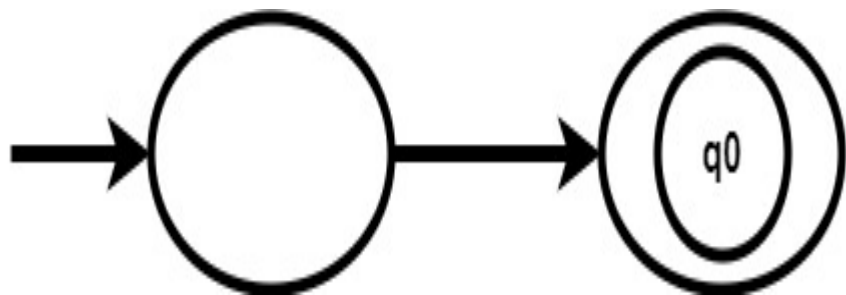
Regular Expression to NFA with E

Statement :- Let r be a regular expression then there exists an NFA with E transition that accept $L(r)$

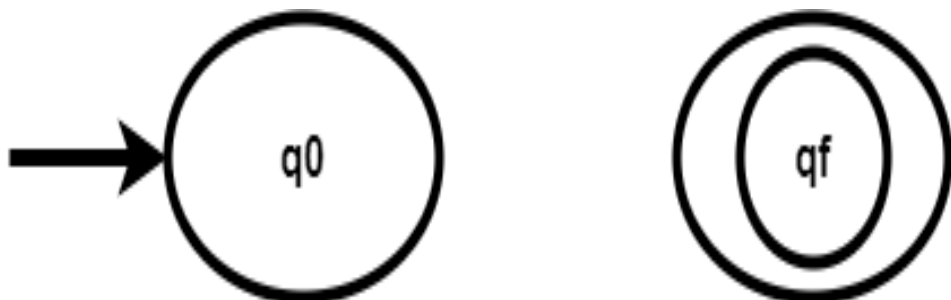
Zero operations:

Expression r must be E , ϕ , a

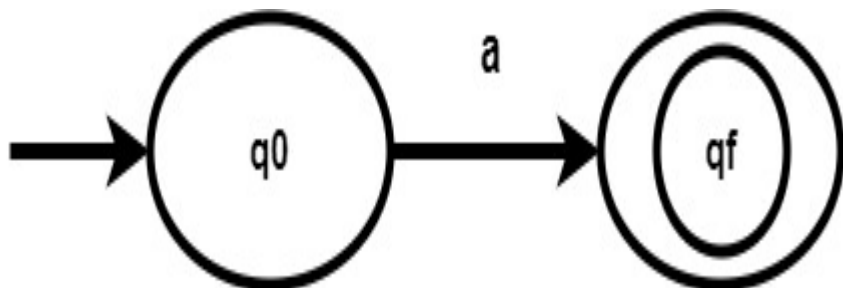
For some a in E .



$$r = E$$



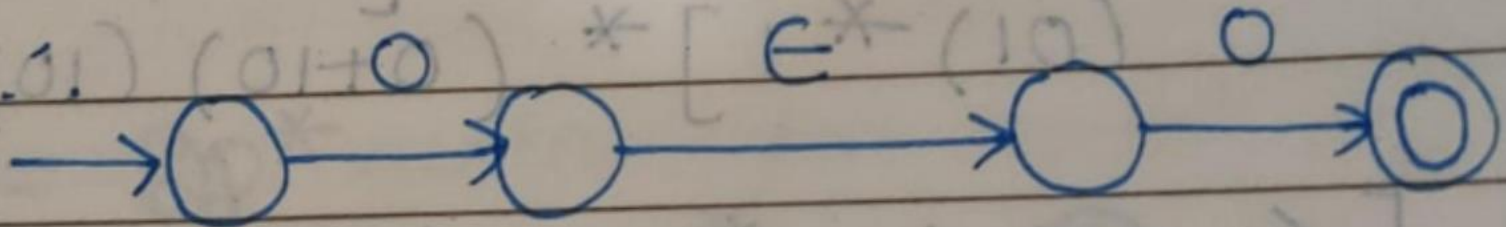
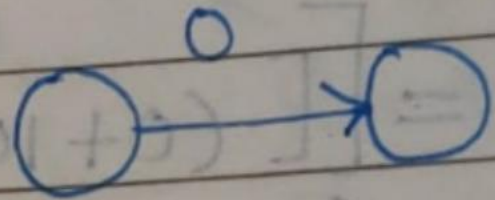
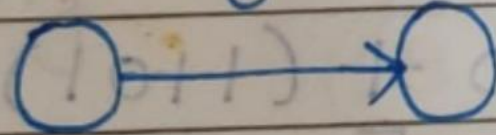
$$r = \phi$$



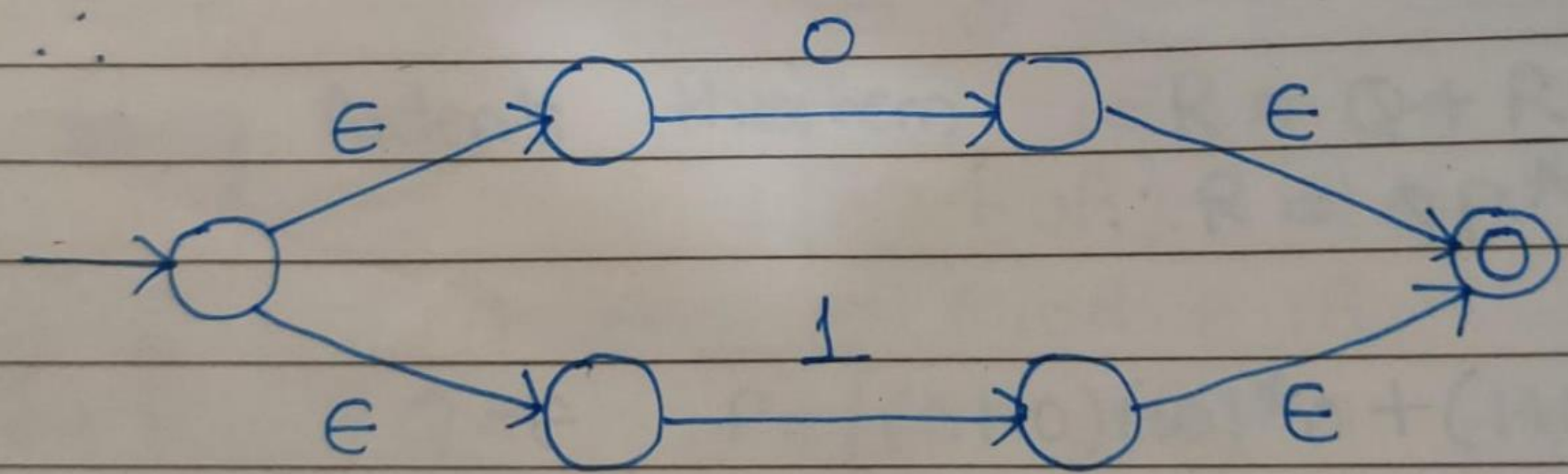
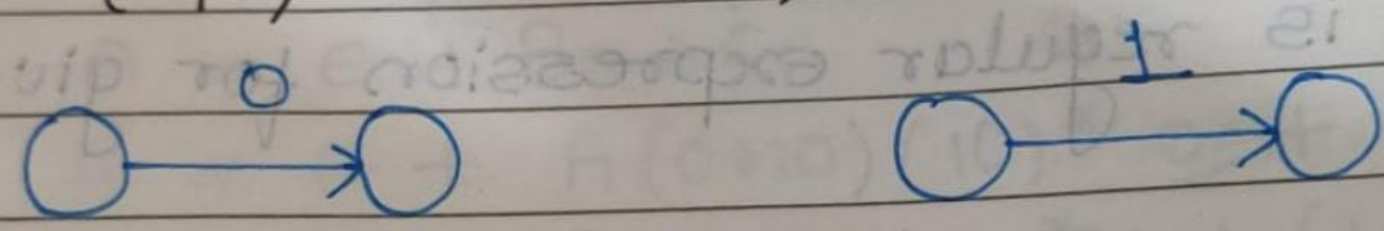
$$r = a$$

Some Basic Cases

① 00 (concatenation)

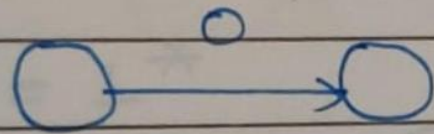


② (0+1) or (0|1) (union)

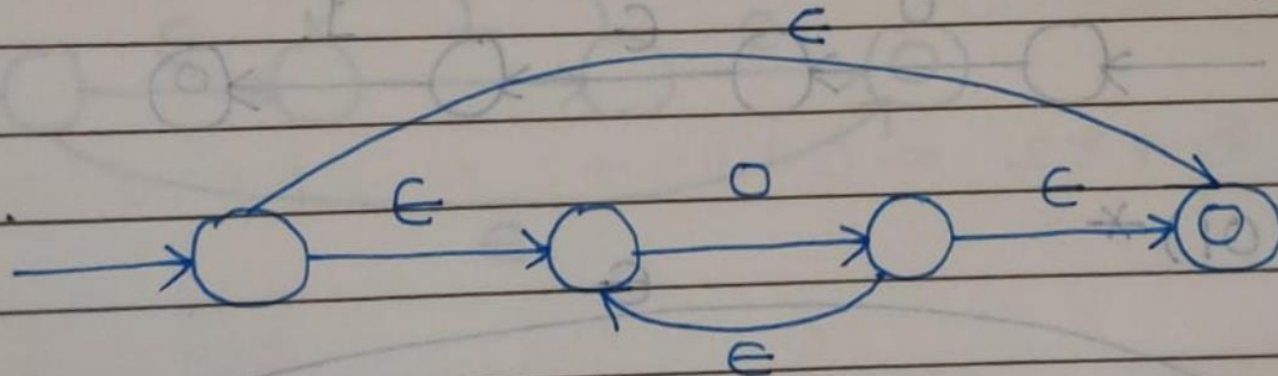


③ 0^* (closure)

$$0^* = \{ \epsilon, 0, 00, 000, \dots \}$$



Now, given initial & final state.



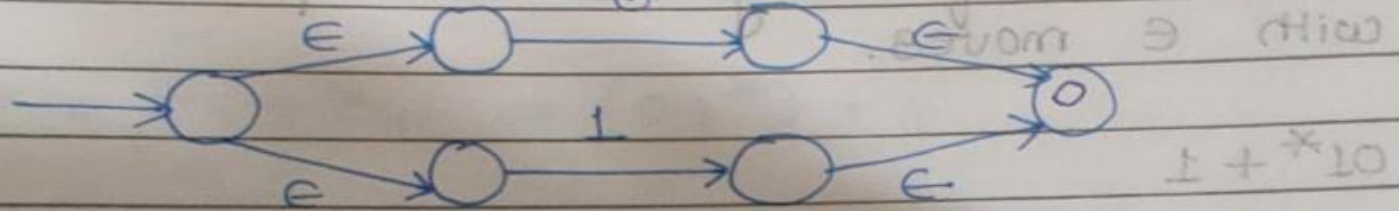
as, $\epsilon 0 \epsilon 0 \epsilon 0$
 $= 000 \dots$

④ $(0+1)^*$ or $(0|1)^*$

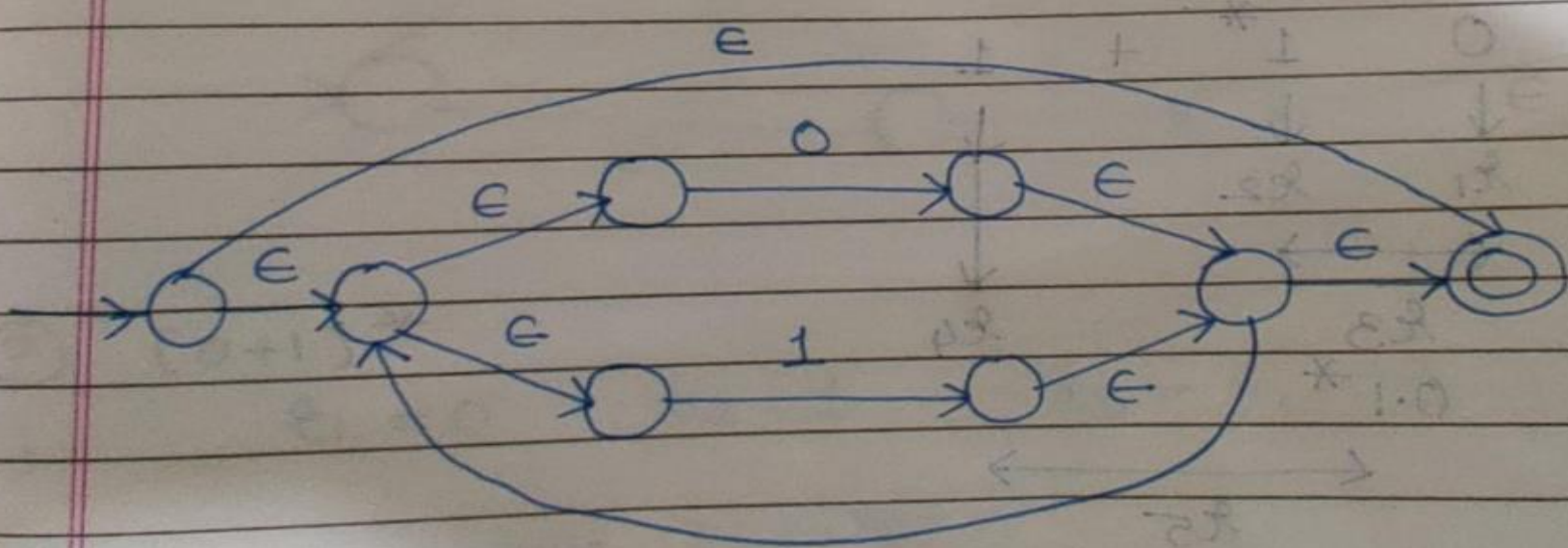
$(0+1)^*$ or $(0|1)^* = \{ \epsilon, 0, 1, (0+1), \dots \}$

For $(0+1)^*$ or $(0|1)^*$

Convert the following RE into equivalent NFA



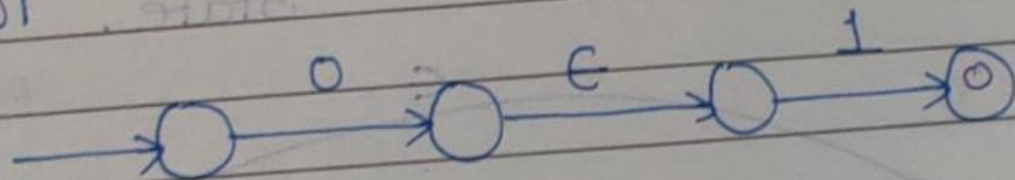
Now, $(0+1)^*$ or $(0|1)^*$



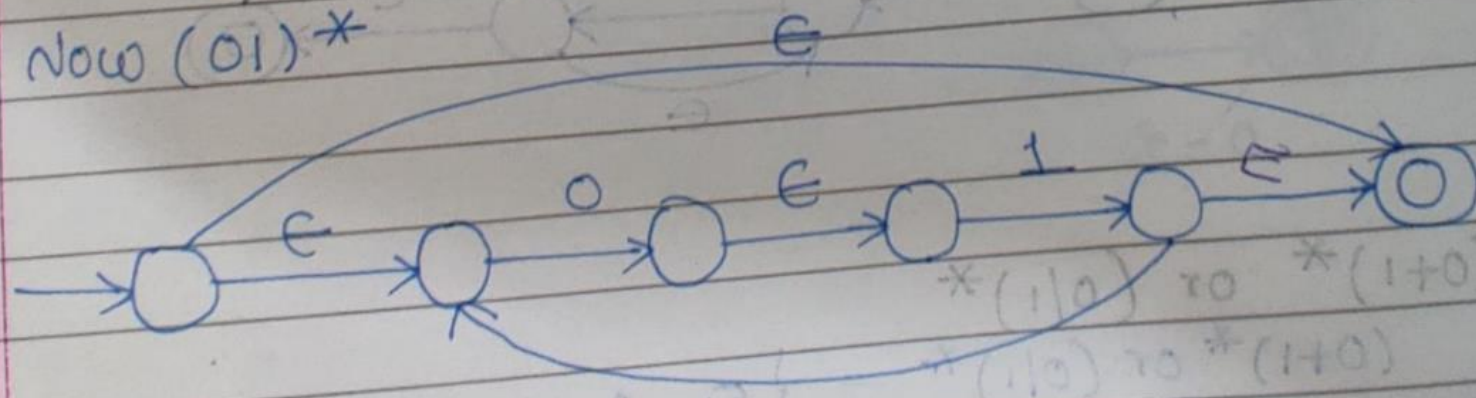
⑤ $(0.1)^*$

$$(0.1)^* = \{ \epsilon, 0, 1, 01, 0101, \dots \}$$

for 01



Now $(01)^*$



Convert the following Regular Expression into equivalent NFA with E moves

$01^* + 1$

Convert the following Regular Expression into equivalent NFA with E moves

$$(0 + 1)^*$$

Convert the following Regular Expression into equivalent NFA with E moves

$$(1 + 01)^*$$

Convert the following Regular Expression into equivalent NFA with E moves

$(0 + 1)^* 011$

Convert the following Regular Expression into equivalent NFA with E moves

$0^*1^*2^*$

Convert the following Regular Expression into equivalent NFA with E moves

$$***a(a + b) * b***$$

Home Work

Convert the following Regular Expression into equivalent NFA with E moves

$10(00 + 11)^* 01$

$(00 + 1)^* (11 + 0)^*$

$20 + (0 + 11)0^* 1$

$(0 + 1)^* (00 + 11)(0 + 1)^*$

$[(0 + 1)(0 + 1)]^* + [(0 + 1)(0 + 1)(0 + 1)]^*$

Regular Expression to NFA without E

Construct NFA equivalent which must be without E of the following RE

$$10 + (0 + 11)0^*1$$

Construct NFA equivalent which must be without E of the following RE

$$(0+1)^*(00+11)(0+1)^*$$

Construct NFA equivalent which must be without E of the following RE

$$(0+1)^*(00+11)(0+1)^*$$

Construct NFA equivalent which must be without E of the following RE

$$abb + ab^* + aab^*ab + aba$$

In formal language theory, a grammar is typically represented using a tuple, especially when defining context-free grammars (CFGs). The standard notation for a grammar is represented as a 4-tuple:

$$G=(V,T,P,S)$$

Where:

- **V** (Variables or Non-terminals): A finite set of non-terminal symbols. These symbols can be replaced or expanded using production rules. They are also referred to as syntactic variables.
- **T** (Terminals): A finite set of terminal symbols, which form the actual content of the strings generated by the grammar. Terminal symbols are the alphabet of the language and appear in the final output strings.
- **P** (Productions or Rules): A finite set of production rules. Each rule is of the form: $A \rightarrow \alpha$

Where:

A is a non-terminal symbol from V .

α is a string of symbols from $(V \cup T)^*$ (i.e., a combination of terminals and/or non-terminals).

- **S** (Start Symbol): A special non-terminal symbol from V that denotes the starting point for generating strings. It is the symbol from which production begins.

Consider a simple grammar G defined as:

$G=(V,T,P,S)$ With the following components:

- $V=\{S,A\}$ (Set of non-terminals)
- $T=\{a,b\}$ (Set of terminals)
- $P=\{S\rightarrow aA, A\rightarrow b\}$ (Set of productions)
- S (Start symbol)

This grammar generates strings starting with **a** and followed by **b**.

P- Production Rules Represented

LHS-----> RHS

Single Non Terminal-----> E | String of terminal & Non terminals.

A-----> aB

B-----> E

A-----> aB c

B-----> E

C-----> b

REGULAR GRAMMER

A Regular Grammar is a formal grammar that is used to generate regular languages. It is a subset of context-free grammars and consists of rules that have specific constraints. Regular grammars are closely related to Regular Expressions and Finite Automata, making them useful for defining patterns in lexical analysis, programming languages, and text processing.

Types of Regular Grammars

There are two main types of regular grammars:

- 1. Right-Linear Grammar**
- 2. Left-Linear Grammar**

1. Right-Linear Grammar

In a Right-Linear Grammar, each production rule must satisfy one of the following forms:

$$A \rightarrow aB$$

$$A \rightarrow a$$

$$A \rightarrow \varepsilon$$

Where:

- A and B are non-terminal symbols.
- a is a terminal symbol.
- ε is the empty string (used to denote termination or end of string).

2. Left-Linear Grammar

In a Left-Linear Grammar, each production rule must satisfy one of the following forms:

$A \rightarrow Ba$

$A \rightarrow a$

$A \rightarrow \epsilon$

Where:

- A and B are non-terminal symbols.
- a is a terminal symbol.
- ϵ is the empty string.

Pumping Lemma for Regular Languages

The Pumping Lemma is a powerful tool for showing that a language is not regular by deriving contradictions using the properties of the lemma.

The **Pumping Lemma** is a fundamental property used to prove that a given language is **not regular**. It provides a necessary condition that every regular language must satisfy. If a language does **not** satisfy this condition, then it cannot be a regular language. The lemma states that if a language is regular, any sufficiently long string in the language can be "pumped" (repeated) in such a way that the resulting string is still in the language.

